

【游戏开发教程】BehaviorDesigner插件制作AI行为树（Unity | 保姆级教程 | 动态图演示 | Unity2021最新版）

文章目录

- 一、前言
- 二、插件下载
 - 1、AssetStore下载
 - 2、GitCode下载
- 三、官方教程
 - 1、在线文档
 - 2、离线文档
- 四、插件界面
 - 1、打开编辑器
 - 2、界面介绍
- 五、快速制作一棵行为树
 - 1、创建物体
 - 2、挂BehaviorTree脚本
 - 3、添加Task节点
 - 4、运行测试
 - 5、导出BehaviorTree
 - 6、手动引用BehaviorTree树资源
 - 6.1、设置External Behavior
 - 6.2、使用Behavior Tree Reference
 - 7、动态加载行为树资源并设置External Behavior
- 六、复合节点：Composities
 - 1、Sequence：顺序节点
 - 1.1、节点介绍
 - 1.2、新发口诀
 - 1.3、案例1：通知开饭了
 - 1.4、案例2：吃几口饭吃一口菜
 - 2、Parallel：并行节点
 - 2.1、节点介绍
 - 2.2、新发口诀
 - 2.3、案例：边跑步边听歌
 - 3、Selector：选择节点
 - 3.1、节点介绍
 - 3.2、新发口诀
 - 3.3、案例：诈骗举报概率
 - 4、Parallel Selector：并行选择节点
 - 4.1、节点介绍
 - 4.2、新发口诀
 - 4.3、案例：考研和找工作
 - 5、Priority Selector：优先级选择节点
 - 5.1、节点介绍
 - 5.2、新发口诀
 - 5.3、案例：先吵架还是先道歉
 - 6、Random Selector：随机选择节点
 - 6.1、节点介绍
 - 6.2、新发口诀
 - 6.3、案例：吃米饭、面条、饺子
 - 7、Random Sequence：随机顺序节点
 - 7.1、节点介绍
 - 7.2、新发口诀
 - 7.3、案例：番茄炒蛋
 - 8、Parallel Complete：并行竞争节点
 - 8.1、节点介绍
 - 8.2、新发口诀
 - 9、Selector Evaluator：评估选择节点
 - 9.1、节点介绍
 - 9.2、新发口诀
- 七、中断
 - 1、Self中断：老妈喊你回家吃饭
 - 2、Lower Priority中断：睡觉时地震

3、Both中断：两者都中断

八、Variables：行为树成员变量

- 1、添加变量
- 2、读取变量
- 3、修改变量
- 4、拓展变量类型
- 5、代码读写变量

九、行为树事件

- 1、Send Event节点
 - 1.1、Target Game Object参数
 - 1.2、Event Name参数
 - 1.3、Group参数
 - 1.4、Argument参数
- 2、Has Receive Event节点
 - 2.1、Event Name参数
 - 2.2、Stored Value参数
- 3、代码中注册和注销事件

十、拓展行为树节点

- 1、继承
- 2、案例：MoveTo
- 3、官方Sample包

十一、完毕

一、前言

嗨，大家好，我是新发。之前用过 Behavior Designer 行为树 插件，最近又需要用到，我发现网上的教程很多写得不够好，今天我就来写写 Behavior Designer 行为树的教程吧，希望可以帮到大家。

二、插件下载

1、AssetStore下载

插件可以在 AssetStore 下载到，

地址：<https://assetstore.unity.com/packages/tools/visual-scripting/behavior-designer-behavior-trees-for-everyone-15277>

目前最新的版本是 1.7.2，

可以在官网查看每个版本的迭代内容：

<https://opsive.com/news/category/release-notes/behavior-designer-release-notes/>

2、GitCode 下载

我放了 1.7.1 版本的到 GitCode 上，地址：<https://gitcode.net/linxinf/unitybehaviordesigner>

下载下来后，放入你的工程即可，

其中我加了一个 FixGUIStyle 脚本，

因为在 Unity 专业版上，打开 Behavior Designer 编辑器 是这个鬼样，

注：如果你使用的是个人版，则不会有这个问题

它的 GUIStyle 在 Unity 专业版本有问题，插件编辑器是个 dll 文件，我们无法修改编辑器源码，

然后我进行了反编译，找到了它的封装各种 GUIStyle 的类： BehaviorDesignerUtility，

通过反射，修改这个类的私有静态成员对象，代码如下：

```
1 using UnityEngine;
2 using UnityEditor;
3 using System.Reflection;
4 using BehaviorDesigner.Editor;
5
```

```
6
7
8 /// <summary>
9 /// 修复BehaviorDesigner编辑器的GUIStyle问题
10 /// </summary>
11 public class FixGUIStyle
12 {
13     [MenuItem("Tools/Behavior Designer/修复界面GUIStyle #b", false, 0)]
14     [RuntimeInitializeOnLoadMethod(RuntimeInitializeLoadType.BeforeSceneLoad)]
15     public static void ShowWindow()
16     {
17
18         var guiStyle = new GUIStyle();
19         guiStyle.alignment = TextAnchor.UpperCenter;
20         guiStyle.fontSize = 12;
21         guiStyle.wordWrap = true;
22         guiStyle.normal.textColor = Color.white;
23         guiStyle.active.textColor = Color.white;
24         guiStyle.hover.textColor = Color.white;
25         guiStyle.focused.textColor = Color.white;
26
27         Texture2D tex2D = new Texture2D(1, 1, TextureFormat.RGB24, false);
28         tex2D.SetPixel(1, 1, new Color(0.1f, 0.1f, 0.1f, 1f));
29         tex2D.hideFlags = HideFlags.HideAndDontSave;
30         tex2D.Apply();
31         guiStyle.normal.background = tex2D;
32
33         // 反射动态修改私有静态成员, 修改GUIStyle
34         var t = typeof(BehaviorDesignerUtility);
35         var graphBackgroundGUIStyle = t.GetField("graphBackgroundGUIStyle", BindingFlags.Static | BindingFlags.NonPublic);
36         graphBackgroundGUIStyle.SetValue(null, guiStyle);
37
38
39         var taskCommentGUIStyle = t.GetField("taskCommentGUIStyle", BindingFlags.Static | BindingFlags.NonPublic);
40         taskCommentGUIStyle.SetValue(null, guiStyle);
41
42
43         var selectedBackgroundGUIStyle = t.GetField("selectedBackgroundGUIStyle", BindingFlags.Static | BindingFlags.NonPublic);
44         selectedBackgroundGUIStyle.SetValue(null, guiStyle);
45
46
47         var preferencesPaneGUIStyle = t.GetField("preferencesPaneGUIStyle", BindingFlags.Static | BindingFlags.NonPublic);
48         preferencesPaneGUIStyle.SetValue(null, guiStyle);
49
50     }
51 }
52
```

AI写代码csharp运行

点击菜单 **Tools/Behavior Designer/修复界面GUIStyle** , 或者按一下 **Shift+B** 快捷键, 即可修复 **GUIStyle** 问题,

效果如下

三、官方教程

学习插件的时候, 建议先看下官方文档。

1、在线文档

官方在线文档: <https://opsive.com/support/documentation/behavior-designer/overview/>

2、离线文档

插件的 **Behavior Designer** 目录中有个 **Documentation.pdf** 文档, 不过可能很多同学都没去看这份文档, 建议还是看一看,

四、插件界面

1、打开编辑器

点击菜单 **Tools/Behavior Designer/Editor** 即可打开编辑器窗口,

2、界面介绍

界面可以划分为 3 个区域，如下

第 1 个区域是行为树的组织区域，我们的行为树节点就是在这个区域上进行连线的；

第 2 个区域顶部是四个标签页： Behavior 、 Tasks 、 Variables 、 Inspector ， 默认是 Tasks 标签页，显示的是节点列表；

标签页	说明
Behavior	对整个行为树的设置，比如行为树名称、是否激活时立即执行、执行完是否是否重新执行等设置
Tasks	任务节点列表，比如Composities（符合节点）、Decorators（装饰节点）、Actions（行为节点）等等
Variables	行为树变量，可以设置全局变量，也可以设置行为树自身的变量
Inspector	查看节点详细信息，设置节点参数

第 3 个区域按钮功能如下

五、快速制作一棵行为树

在讲解节点之前，我先演示一遍如何快速制作一棵行为树，让大家对整个流程有个概念。

1、创建物体

首先在场景中创建一个物体，可以是任何物体，比如你想控制角色，那么这个物体就可以是你的角色模型，这里我创建一个空物体，

重命名为 BehaviorTreeObj ，

2、挂BehaviorTree脚本

打开 Behavior Designer 编辑器，先选中刚刚的 BehaviorTreeObj 物体，然后在编辑器网格空白处右键鼠标，点击菜单 Add Behavior Tree ，

这一步其实就是给物体挂上 BehaviorTree 组件，我们也可以通过 Add Component 手动添加 BehaviorTree 组件，

3、添加Task节点

我以最简单的 Log 节点为例，它的功能就是输出日志，在空白处点击鼠标左键，按键盘的 空格键 即可弹出节点搜索框，搜索 log ， 可以看到 Log 节点，点击即可创建出 Log 任务节点，它会自动帮我添加一个 Entry 入口节点， Entry 连向 Log ， 如下

我们给 Log 节点设置 Text 字段的值为 Hello World ， 并添加注释，如下

4、运行测试

运行 Unity ， 可以看到 Log 节点被执行了，显示了一个绿色的勾，日志窗口也输出了 HelloWorld ，

5、导出BehaviorTree

点击 Export 按钮，将行为树导出保存为资源文件，

如下，这样子就可以复用行为树资源了，

6、手动引用BehaviorTree树资源

6.1、设置External Behavior

我们把上面导出的行为树资源赋值给 BehaviorTree 组件的 External Behavior 成员，如下

运行时，就会根据这个 External Behavior 的行为树来执行了。

6.2、使用Behavior Tree Reference

另外，我们还可以通过 Behavior Tree Reference 节点来引用行为树资源，
先创建一个 Behavior Tree Reference 节点，点击 Inspector，设置它引用的外部行为树资源即可，

一般我们会做一个通用的行为树保存为资源文件，然后对特殊的怪物另外做行为树，并让其引用通用行为树资源，像这样子

7、动态加载行为树资源并设置External Behavior

我们也可以通过代码动态设置 External Behavior，我这里使用 Addressables 插件来做资源加载，

注：关于 Addressables 的使用，可以参见我之前写的这篇文章：[【游戏开发探究】Unity Addressables资源管理方式用起来太爽了，资源打包、加载、热更变得如此轻松（Addressable Asset System | 简称AA）](#)

首先给行为树资源文件添加 Addressable 管理，分配到一个 Addressable 索引名，如下

注：勾上Addressable就会自动分配一个索引名了

接着我们可以使用代码加载行为树资源，动态赋值给 BehaviorTree 组件的 ExternalBehavior 对象，
创建一个 AddBehaviorTree.cs 脚本，

代码如下

```
1 using System.Collections;
2 using UnityEngine;
3 using BehaviorDesigner.Runtime;
4 using UnityEngine.AddressableAssets;
5
6 public class AddBehaviorTree : MonoBehaviour
7 {
8     IEnumerator Start()
9     {
10         // 初始化Addressables
11         yield return Addressables.InitializeAsync();
12         // 添加BehaviorTree
13         var bt = gameObject.AddComponent<BehaviorTree>();
14         // 设置不立即执行
15         bt.StartWhenEnabled = false;
16         // 加载行为树资源
17         var loader = Addressables.LoadAssetAsync<ExternalBehaviorTree>("Assets/BtRes/Behavior.asset");
18         yield return loader;
19         // 设置ExternalBehavior
20         bt.ExternalBehavior = loader.Result;
21         // 执行行为树
22         bt.EnableBehavior();
23     }
24 }
```

AI写代码csharp运行

我们创建一个空物体，重命名 BtObj，并给它挂上 AddBehaviorTree 脚本，如下

我们运行 Unity，可以看到 BtObj 物体动态挂了 BehaviorTree 组件，并且动态设置了 ExternalBehavior 对象，行为树也被执行了，输出了 Hello World，如下

我们可以点击 BehaviorTree 组件上的 Open 按钮，

可以看到行为树已经执行完毕，

如果我们想让行为树执行完毕后重复执行，可以设置 RestartWhenComplete 为 true，

我们串一个 Wait 节点，方便观察，

执行效果如下，可以看到行为树可以重复执行了，

上面我们用到了Sequence顺序节点，它是一个复合节点，下文我会进行节点的详细介绍。

六、复合节点：Composities

行为树任务节点中最重要的节点就是复合节点了，它决定了行为树的执行流，也就是Tasks列表中Composities分组的这些节点，

我先简单做个日常生活的行为树给大家看看，如下

上面的行为树的执行过程是怎么样的呢？想要搞清楚就得先知道行为树的执行顺序：从左到右，并且深度优先。
另外，复合节点又控制着子节点的执行规则，是顺序执行还是并行执行，亦或者随机执行，是遇到成功就立即返回成功还是遇到失败就立即返回失败，亦或者遇到成功继续执行下一个，或者遇到失败继续执行下一个呢？
这些都是复合节点来控制的，这里头的逻辑需要大家好好琢磨，下面我带大家挨个过一遍复合节点，讲解过程中我会穿插顺带讲解一下用到的其他任务节点，并举一些实际的应用场景，方便大家学以致用，也希望大家能够触类旁通。

1、Sequence：顺序节点

1.1、节点介绍

顾名思义，顺序节点就是从左到右挨个顺序执行，当所有子节点都返回Success时，它才返回Success，

当某个子节点返回Failure时，顺序节点就会立刻返回Failure，

1.2、新发口诀

顺序节点，从左到右，为真继续，全真才真，一假即假，一假即停。
类似于与逻辑。

1.3、案例1：通知开饭了

我们使用Has Received Event来监听开饭的事件，当它检测到事件时才会返回Success，此时顺序节点就会去执行下一个子节点，即干饭的逻辑，达到响应事件的功能，如下

其中Has Received Event节点要设置监听的事件名称，定义一个字符串即可，

抛出事件有两种方式，一种是代码调用行为树的SendEvent接口，如下

```
1 public void SendEvent(string name);
2 public void SendEvent<T>(string name, T arg1);
3 public void SendEvent<T, U>(string name, T arg1, U arg2);
4 public void SendEvent<T, U, V>(string name, T arg1, U arg2, V arg3);
AI写代码csharp运行
```

示例：

```
1 // bt是BehaviorTree对象
2 bt.SendEvent("EAT_EVENT");
AI写代码csharp运行
```

另外一种是使用SendEvent节点，

设置要发送的事件名称即可，

1.4、案例2：吃几口饭吃一口菜

想象一下你吃饭的过程，扒拉吃几口饭，然后吃一口菜，我们可以用顺序节点来组织，

2、Parallel：并行节点

2.1、节点介绍

并行节点，顾名思义，它会并行执行所有的子节点，所有的子节点都返回Success，并行节点才会返回Success，

只要有一个子节点返回Failure，并行节点就会立刻返回Failure。如下图，我们可以看到，等10秒和等20秒的两个节点还没执行完就被停止了，因为中间那个等1秒

2.2、新发口诀

并行节点，并行执行，全真才真，一假即假，一假即停，被停即假。

2.3、案例：边跑步边听歌

比如我们边跑步边听歌，同时还要监听是否有下雨，有下雨的话要中断跑步，如下

这个例子我用到了 **Selector** 节点，并且启用了 **Self** 中断，看不懂的同学不要着急，下文我会讲。

3、Selector：选择节点

3.1、节点介绍

选择节点与顺序节点类似，也是从左到右执行，不同的是，当有子节点返回 **Success**，选择节点就会立刻返回 **Success**，不会执行下一个子节点

如果子节点返回 **Failure**，会就执行下一个子节点，其实很好理解，就是从左到右遍历选一个 **Success**，有 **Success** 就返回 **Success**，否则执行下一个子节点，

如果所有子节点都返回 **Failure**，选择节点才返回 **Failure**，

3.2、新发口诀

选择节点，从左到右，一真即真，一真即停，全假才假。

3.3、案例：诈骗举报概率

比如收到诈骗短信后，有 **99%** 的概率会执行举报，有 **1%** 的概率会上当受骗，如下

4、Parallel Selector：并行选择节点

4.1、节点介绍

顾名思义，就是并行执行选择节点，有一个 **Success** 就立即返回 **Success**，并且会终止其他子节点，被终止的子节点会返回 **Failure**，如下

所有子节点都返回 **Failure** 时，并行选择节点才返回 **Failure**，

4.2、新发口诀

并行选择节点，并行执行，一真即真，一真即停，全假才假。

4.3、案例：考研 和找工作

现在人生道路有两条路：考研、找工作，同时进行，其中一件事情成了，就吃一顿好的庆祝一下，如下

5、Priority Selector：优先级选择节点

5.1、节点介绍

这个节点和 **Selector** 类似，**Selector** 的执行顺序是从左到右，而 **Priority Selector** 会先检查子节点的优先级（**priority**）进行排序，优先级高的优先执行。问题来了，我们如何设置子节点的优先级呢？节点的优先级默认是 **0**，如果要修改优先级，需要重写 **Task** 的 **GetPriority** 方法，

这里我封装一个带优先级参数的日志节点吧：**PriorityLog**，创建一个 **PriorityLog** 脚本，代码如下

```
1 using UnityEngine;
2
3 namespace BehaviorDesigner.Runtime.Tasks
4 {
5     [TaskDescription("带优先级参数的日志节点")]
6     [TaskIcon("{SkinColor}LogIcon.png")]
7     public class PriorityLog : Action
8     {
9         [Tooltip("Text to output to the log")]
10
```

```
10 public SharedString text;
11 [Tooltip("Is this text an error?")]
12 public SharedBool logError;
13 [Tooltip("Should the time be included in the log message?")]
14 public SharedBool logTime;
15
16 [Tooltip("优先级")]
17 public SharedFloat priority;
18
19 /// <summary>
20 /// 重写获取优先级的方法
21 /// </summary>
22 /// <returns></returns>
23 public override float GetPriority()
24 {
25     return priority.Value;
26 }
27 public override TaskStatus OnUpdate()
28 {
29     // Log the text and return success
30     if (logError.Value) {
31         Debug.LogError(logTime.Value ? string.Format("{0}: {1}", Time.time, text) : text);
32     } else {
33         Debug.Log(logTime.Value ? string.Format("{0}: {1}", Time.time, text) : text);
34     }
35     return TaskStatus.Success;
36 }
37
38 public override void OnReset()
39 {
40     // Reset the properties back to their original values
41     text = "";
42     logError = false;
43     logTime = false;
44 }
45 }
46 }
```

AI写代码csharp运行

现在我们就可以在 **Tasks** 列表中看到我们新封装的 **PriorityLog** 节点了，

使用它连个简单的行为树，如下

我们可以选中节点，在 **Inspector** 中设置节点的 **Priority**，

由于右边的节点优先级我们设置为了 **1**，比左边的节点优先级高，所以会优先执行右边的节点，因为右边的节点执行返回了 **Success**，此时 **Priority Selector** 就会直接返回 **Success**，不会去执行左边的节点了，如下

5.2、新发口诀

优先级选择节点，优先级排序，一真即真，一真即停，全假才假。

5.3、案例：先吵架还是先道歉

我先给行为树创建一 **Float** 类型的变量：**SorryPriority**，如下

以它作为道歉优先级变量，

接着制作如下的行为树，当按下 **A** 键的时候，设置 **SorryPriority** 变量，提高道歉的优先级，

其中 **Set Shared Float** 节点的设置如下，让它去修改 **SorryPriority** 变量为 **1**，

道歉节点的 **Priority** 参数设置为 **SorryPriority** 变量，如下

我们运行，如果我没有按下 **A** 键，则道歉优先级是 **0**，2 秒情绪酝酿完毕后，会默认先执行吵架，

如果我在 2 秒情绪酝酿期间按下了 **A** 键，则会设置道歉优先级变量为 **1**，情绪酝酿完毕后就会优先执行道歉，就不会吵架了，

6、Random Selector: 随机选择节点

6.1、节点介绍

这个很好理解，按选择节点的规则，只是不是从左到右执行，而是随机顺序执行。
如果有子节点返回 **Success**，则立即返回 **Success**，否则继续执行下一个节点，

只有所有的子节点都返回 **Failure** 时，**Random Selector** 才返回 **Failure**，

6.2、新发口诀

随机选择节点，随机执行，一真即真，一真即停，全假才假。

6.3、案例：吃米饭、面条、饺子

中午吃什么好呢？随机选一个吧，好的，吃面条~

7、Random Sequence：随机顺序节点

7.1、节点介绍

这个也很好理解，就是顺序节点的规则，但不是从左到右执行，而是随机顺序执行。

当所有子节点都返回 **Success** 时，它才返回 **Success**，

当某个子节点返回 **Failure** 时，顺序节点就会立刻返回 **Failure**，

7.2、新发口诀

随机顺序节点，随机顺序，为真继续，全真才真，一假即假，一假即停。

7.3、案例：番茄炒蛋

我现在要做一道番茄炒蛋，有的人先炒番茄，有的人先炒鸡蛋，这里我们就可以使用随机顺序节点了，如果炒鸡蛋或者炒番茄失败，那么我们就吃不到番茄炒蛋了，只有两个都成功了，才能吃到番茄炒蛋，

8、Parallel Complete：并行竞争节点

8.1、节点介绍

这个节点其实就是并行执行所有的子节点，只要有一个子节点返回了结果，它就结束，并以这个子节点的结果为结果，

8.2、新发口诀

并行竞争节点，并行执行，最速之子，以子为果。

9、Selector Evaluator：评估选择节点

9.1、节点介绍

这个节点不是很好理解，它的逻辑是这样的，从左到右顺序执行，如果遇到子节点返回 **Success**，则立即结束，并返回 **Success**，否则继续执行下一个子节点。

有同学会问了，那它与 **Selector** 节点有啥区别？

上面我用的是 **Log** 节点，会立即返回结果，如果是用 **Wait** 节点，则会有 **Running** 状态，这个 **Selector Evaluator** 节点是个固执的强迫症节点，它认为子节点应该给它返回 **Success**，如果子节点返回了 **Failure**，它会硬着头皮执行下一子节点，但是，一旦下一个子节点处于 **Running** 状态，它就会像个渣男一样毫不犹豫地打断并返回第一个子节点，重新执行，只要有机会它就想要一个 **Success** 结果，如果后续的子节点都是立即返回成功或失败，而没有 **Running** 状态，则它会接受后续子节点的 **Success**，也就是说，它也不是非得第一个子节点不娶，后面的子节点只要立刻 **Success**，它就不打断去重新找第一个子节点了，相当地渣男啊！

看，第二任只要考虑一下（**Running** 状态），它就会立即打断回去执行初恋，

如果第二任立即答应了，它就会接受第二任的结果，

如果第二任直接给个 **Failure**，那么这个渣男会去骚扰路人甲，如果路人甲给它 **Success**，它就会爱上路人甲，

如果路人甲立即给它 **Failure**，这个渣男才会死心，

9.2、新发口诀

评估选择节点，渣男一个，从左到右，遇真即真，遇假继续，**Runing** 时机打断，从头来过。

七、中断

复合节点有中断的权利，我们可以选中某个复合节点，然后点击 **Inspector**，设置它的中断模式，

默认为 **None**，即无中断，另外三个中断模式，下文我会挨个进行演示。

这里我先解释一下什么是中断，比如你正在睡觉，突然地震了，这个时候睡觉就要被中断，执行逃跑的逻辑，命要紧。

复合节点的中断控制是由谁来调度的呢？细心的同学应该会发现，执行行为树的时候，会自动创建一个 **Behavior Manager** 对象，上面挂着 **BehaviorManager** 脚本，我们的行为树的执行都是由这个 **BehaviorManager** 来调度的，它就是最高司令，

感兴趣的同学可以反编译 **BehaviorDesigner.Runtime.dll**，查看 **BehaviorManager** 的源码，

1、Self中断：老妈喊你回家吃饭

一个复合节点下可以有多个子节点，假设现在有两个子节点，假设此时正在执行第二个子节点（**Running** 状态），突然来了一个事件，需要中断第二个子节点，立即执行第一个子节点，这里就可以使用 **Self** 中断，**Self** 其实是站在复合节点的角度的，就是中断复合节点自己。

假设你去朋友家打游戏，你决定打完游戏就回家，如果打游戏过程中，老妈打电话喊你回家吃饭，那么就中断打游戏，回家。

行为树如下

没有按下 **A** 键的执行效果如下，打完游戏开开心心回家，

打游戏过程中，按下 **A** 键，中断打游戏，回家，

2、Lower Priority中断：睡觉时地震

我们以睡觉的时候发生地震为例，演示一下 **Lower Priority** 中断。

睡觉的时候如果发生地震，需要中断睡觉，执行逃命逻辑。

如下，睡觉节点与**Sequence**节点是平级关系，并且睡觉节点在右边，属于低优先级，当正在执行睡觉节点时，左边的**Sequence**节点想要中断右边的睡觉节点，设置 **Lower Priority** 中断即可，

我们从中断的图标也可以看出，它指向右边，表示中断右边的节点的意思，

3、Both中断：两者都中断

Both 中断其实就是 **Self** 和 **Lower Priority** 都中断。如下

八、Variables：行为树成员变量

我们点击 **Variables**，可以给行为树添加成员变量，也可以查看已添加的成员变量，

1、添加变量

先起一个变量名，然后选择变量的数据类型，我们也可以添加自定义的数据类型，需要进行变量类型拓展，下文会讲解拓展变量类型的步骤，这里我们先选一个 **String** 好了，最后点击 **Add** 按钮即可，

我们可以修改变量名，给变量设置初始值，如下

2、读取变量

现在我们想通过 **Log** 节点把 **myName** 这个变量输出出来。

先选中 **Log** 节点，切到 **Inspector** 页，点击 **Text** 值最右边的小点点，如下

此时会出现红色的感叹号

我们点击下拉框，选中 **myName** 变量即可，

我们运行行为树，可以看到日志输出了变量值，如下

3、修改变量

我们在 **Log** 节点左边串一个 **Set String** 节点，

注：如果你的变量类型是 **Bool**，则这里要使用 **Set Bool** 节点，其他类型同理，选择对应的 **Set** 节点，如果是自定义的数据类型，则自己要拓展一个 **Set** 变量值的节点才行。

此时带有一个红色感叹号，我们需要给它设置要修改的变量，如下

我们运行行为树，可以看到输出的变量值是皮皮猫了，

4、拓展变量类型

实际项目中，我们大概率是要拓展行为树的变量类型的，比如我现在有个 **Blog** 类，

```
1  /// <summary>
2  /// 博客
3  /// </summary>
4  public class Blog
5  {
6      /// <summary>
7      /// 作者
8      /// </summary>
9      public string author;
10
11     /// <summary>
12     /// 博客地址
13     /// </summary>
14     public string url;
15
16     public override string ToString()
17     {
18         return $"author: {author}, url: {url}";
19     }
20 }
```

AI写代码csharp运行

我想让他出现行为树 **Variables** 的变量类型列表中，我们需要给它包装一层 **SharedBlog**。

新建一个 **SharedBlog** 脚本，

按照下面这个格式写即可：

```
1  namespace BehaviorDesigner.Runtime
2  {
3      [System.Serializable]
4      public class SharedT : SharedVariable<T>
5      {
6          public static implicit operator SharedT(T value)
7          {
8              return new SharedT { mValue = value };
9          }
10     }
11 }
```

AI写代码csharp运行

SharedBlog 代码如下：

```
1
2  namespace BehaviorDesigner.Runtime
3  {
4      [System.Serializable]
5      public class SharedBlog : SharedVariable<Blog>
6      {
7          public static implicit operator SharedBlog(Blog value)
8          {
9              return new SharedBlog { mValue = value };
10         }
11     }
12 }
```

```
    }  
}  
}  
AI写代码csharp运行
```

此时，我们就可以在变量类型列表中看到我们拓展的 **Blog** 变量类型了，

我们给行为树添加一个 **Blog** 变量吧，

5. 代码读写变量

上面我们拓展了一个 **Blog** 变量类型，并给行为树添加了一个 **Blog** 变量，现在我们在代码中对这个变量进行访问和修改，我们可以通过 **BehaviorTree** 的 **GetVariable** 方法和 **SetVariable** 方法，

```
1 public SharedVariable GetVariable(string name);  
2 public void SetVariable(string name, SharedVariable item);  
AI写代码csharp运行
```

例：

```
1 // 获取行为树对象  
2 var bt = GetComponent<BehaviorTree>();  
3  
4 // 访问成员变量-----  
5 var sharedBlog = (SharedBlog)bt.GetVariable("myBlog");  
6 Debug.Log(sharedBlog.Value.ToString());  
7  
8 // 修改变量值-----  
9 sharedBlog.Value.author = "Unity3D技术博主：林新发";  
10 bt.SetVariable("myBlog", sharedBlog);  
11  
12  
13 // 构建新的成员对象-----  
14 SharedBlog newSharedBlog = new SharedBlog();  
15 Blog newBlog = new Blog();  
16 newBlog.author = "博客技术专家：林新发";  
17 newBlog.url = "https://blog.csdn.net/linxinf";  
18 newSharedBlog.SetValue(newBlog);  
19 // 赋值成员变量  
20 bt.SetVariable("myBlog", newSharedBlog);  
AI写代码csharp运行
```

九、行为树事件

上文讲**Sequence**节点的时候已经举例讲了行为树事件，不过有些细节没有讲到，这里我进行补充。

与事件相关的节点是 **Send Event** 和 **Has Received Event** ,

另外，我们也可以通过代码来发出事件和监听事件，下面给大家演示一遍。

1、Send Event节点

我们首先看 **Send Event** 节点，可以在 **Inspector** 面板看到它的参数，

1.1、Target Game Object参数

行为树之间的事件是可以跨物体发送的，比如 **A** 物体上的行为树给 **B** 物体上的行为树发送事件。

默认为 **None** , 事件会发给自身的行为树。

我们可以查看 **SendEvent** 节点的源码，它调用了 **GetDefaultGameObject** 方法去获取目标物体，

这个方法的源码如下：

意思就是如果为空，就返回自身的 **gameObject** , 也就是行为树自身的物体。

如果我们想要设置这个 **SendEvent** 节点的 **Target Game Object** 参数，可以通过行为树变量来传递，比如在 **Variables** 中定义一个 **targetObj** ,

然后 **SendEvent** 的 **Target Game Object** 引用这个 **targetObj** 变量，

而这个 **targetObj** 变量的赋值，我们可以通过代码来赋值，例：

```
1 // go是目标物体GameObject对象, bt是自身行为树对象
2 SharedGameObject targetObj = new SharedGameObject();
3 targetObj.Value = go;
4 bt.SetVariable("targetObj", targetObj);
    AI写代码csharp运行
```

1.2、Event Name参数

Event Name 就是事件名称，自己定义一个字符串即可，建议为 XXX_EVENT。

1.3、Group参数

这个 Group 参数又是干啥用的呢？因为一个物体可以挂多个 BehaviorTree 组件，假设挂了 3 个 BehaviorTree 组件，我只想给其中的两个 BehaviorTree 发送事件，这个时候就要用到 Group 了。

我们可以在 BehaviorTree 组件上看到有一个 Group 参数，

我们可以给这些 BehaviorTree 分组，默认 Group 是 0，

如果 SendEvent 的 Group 也是 0，那么这个物体下所有 Group 为 0 的 BehaviorTree 都会收到事件。

1.4、Argument参数

我们可以看到有三个 Argument 参数，一般是用来做跨行为树发送事件时的数据传递用的。

比如 A 打了 B，B 收到被打事件，但它并不知道是谁打它的，我们可以给它们定义一个 playerId 成员变量，

注：这里你使用 GameObject 变量也可以，看具体需要~

发事件的时候把 playerId 传递过去，B 在收到事件后会去读取这个参数，B 就知道是 A 打了它了，

2、Has Receive Event节点

Has Receive Event 节点参数如下，

2.1、Event Name参数

Event Name 参数就是要监听的事件名称，与 SendEvent 的 Event Name 要对应上。

2.2、Stored Value参数

Stored Value 参数用来存储传递过来的数据的，与 Send Event 的三个 Argument 顺序对应。注意，这里是把传递过来的参数存储到 Stored Value 设置的变量中，比如 A 行为树传递了一个 playerId 过来，表示打人者的 Id，B 行为树收到事件，把这个 playerId 存储到 fromPlayerId 变量中，

然后我们可以在代码中通过 B 行为树的 GetVariable 方法读取到这个 fromPlayerId，

```
1 var fromPlayerId = (SharedInt)bt.GetVariable("fromPlayerId");
2 Debug.Log($"{fromPlayerId} 打了我");
    AI写代码csharp运行
```

3、代码中注册和注销事件

有时候我们需要在代码中自己注册事件的响应函数，可以调用行为树的 RegisterEvent 方法，

```
1 public void RegisterEvent<T, U, V>(string name, Action<T, U, V> handler);
2 public void RegisterEvent<T, U>(string name, Action<T, U> handler);
3 public void RegisterEvent<T>(string name, Action<T> handler);
4 public void RegisterEvent(string name, System.Action handler);
    AI写代码csharp运行
```

对应的，要注销事件的响应函数，则调用 UnregisterEvent 方法，

```
1 public void UnregisterEvent<T, U, V>(string name, Action<T, U, V> handler);
2 public void UnregisterEvent<T, U>(string name, Action<T, U> handler);
3 public void UnregisterEvent<T>(string name, Action<T> handler);
4 public void UnregisterEvent(string name, System.Action handler);
    AI写代码csharp运行
```

可以查阅 Has Receive Event 节点的源码

十、拓展行为树节点

1、继承

实际项目开发过程中，我们一般都需要自己拓展一些行为树节点，我们知道，所有的节点都是继承 **Task** 的，然后根据节点的功能分类，有细分了一些子类出来，比如复合节点的继承关系如下：

我们想要封装一个新的复合节点，也继承 **Composite** 然后进行拓展即可。

同理，**Action** 类的节点也如此，

我们可以参考 **Log** 节点、**Wait** 节点的写法，自行拓展自己的 **Action** 节点。

2、案例：MoveTo

举个例子，现在要写一个移动主角到指定 **Transform** 位置的 **Action**，首先创建一个 **MoveTo** 脚本，

引入 **BehaviorDesigner** 的命名空间，然后继承 **Action**，

```
1 using UnityEngine;
2 using BehaviorDesigner.Runtime;
3 using BehaviorDesigner.Runtime.Tasks;
4
5 // 移动主角到指定Transform位置
6 public class MoveTo: Action
7 {
8 }
```

AI写代码csharp运行

接着定义一些必要的 **public** 成员变量，这些变量会在 **Inspector** 面板中显示，

```
1 using UnityEngine;
2 using BehaviorDesigner.Runtime;
3 using BehaviorDesigner.Runtime.Tasks;
4
5 // 移动主角到指定Transform位置
6 public class MoveTo : Action
7 {
8     public float speed = 0;
9     public SharedTransform target;
10
11 }
```

AI写代码csharp运行

再接着，我们重写 **OnUpdate** 方法，如下

```
1 public override TaskStatus OnUpdate()
2 {
3     if (Vector3.SqrMagnitude(transform.position - target.Value.position) < 0.1f) {
4         return TaskStatus.Success;
5     }
6     transform.position = Vector3.MoveTowards(transform.position, target.Value.position, speed * Time.deltaTime);
7     return TaskStatus.Running;
8 }
```

AI写代码csharp运行

这样，我们自己封装的 **MoveTo** 节点就完成了，

我们连个行为树，让 **Cub** 来回移动，效果如下

3、官方Sample包

BehaviorDesigner 官方自己写了一些拓展包，我们可以把对应的包导入项目中使用，

这里分享给大家，

链接：<https://pan.baidu.com/s/1mQUkxbeokOh6oyTmEG837w>

提取码：oi74

十一、完毕

好了，就先写到这里吧，我是林新发：<https://blog.csdn.net/linxinfo>

一个在小公司默默奋斗的 Unity 开发者，希望可以帮助更多想学 Unity 的人，共勉~